

Your Name

Your Address in Short

Mobile: +91- ----- Email: -----@gmail.com

Career Objective

Aspiring Python Developer & Machine Learning Engineer with expertise in automation, data science, and deep learning. Skilled in designing intelligent systems, building automation scripts, and applying ML/DL models for real-world problem solving. Seeking opportunities to contribute to impactful projects using modern AI technologies.

Educational Qualification :

Standard	Institute	Board / University	Percentage	Class
MCS				
BCS				
HSC				
SSC				

Technical Skills :

Programming Paradigms & Languages

- **Procedural Programming:** C
- **Object-Oriented Programming (OOP):** C++, Python
- **Virtual Machine-based Programming:** Java, C# .NET
- **Scripting Languages:** PHP, JavaScript

Domain Expertise

- **Python Libraries & Tools:** NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, Keras, OpenCV
- **Machine Learning:** Regression, Classification, Clustering, Feature Engineering, Model Evaluation
- **Deep Learning:** ANN, CNN, RNN, Computer Vision, NLP
- **Automation Tools:** OS, Scripting, Email Automation (smtplib), Scheduling, Logging
- **GenAI & LLMs:** Hugging Face Transformers, GPT/BERT, DALL-E
- **Databases:** MySQL, MongoDB
- **Version Control:** Git
- **Development Environment:** Anaconda, Jupyter Notebook, VS Code

Python Automation Project

Duplicate File Cleaner & Log Automation

Description: Developed a Python-based automation script to detect and delete duplicate files from a directory periodically. The system automatically generates log files of operations performed and shares them via email for audit purposes.

- Implemented checksum-based duplicate detection using hashlib (MD5).
- Automated log generation with timestamps for every execution.
- Used schedule library for periodic execution of duplicate file cleanup.
- Integrated email automation (smtplib) to send logs automatically.

Tech Stack: Python, OS module, Hashlib, Schedule, smtplib

GitHub Repository: github.com/username/-----

System Process Logger with Scheduling

Description: Designed a Python automation project to periodically log details of running processes (PID, name, user, memory usage) on the system. Each execution generates a new log file with a timestamped filename for easy tracking.

- Implemented process scanning using psutil to extract process details.
- Automated log file creation with timestamps and proper formatting.
- Used the schedule library to run logging tasks periodically without manual intervention.
- Enhanced usability by allowing custom folder name & interval input.

Tech Stack: Python, psutil, OS, Schedule, Time

GitHub Repository: github.com/username/-----

Machine Learning Projects

Worked on real-world ML projects covering classification, regression, and clustering.

- **Titanic Survival Predictor** – Built a classification model to predict passenger survival using Decision Trees & Logistic Regression.
- **Diabetes Detection** – Developed a health classification system using medical data to detect diabetes.
- **Breast Cancer Detection** – Achieved high accuracy using Logistic Regression & Support Vector Machine (SVM).
- **Ad Click Predictor** – Classification model predicting probability of user clicks on advertisements.
- **Wine Classifier** – Multi-class classification of wine type using KNN & Random Forest.
- **Head Brain Classification** – Built regression/classification models to analyze the correlation between brain size and intelligence.
- **House Price Prediction** – Regression-based project to predict property prices using multiple features.
- **Iris Classifier** – Implemented classic ML classification on the Iris dataset using Decision Tree & KNN.
- **Advertisement Predictor** – Classification system predicting advertisement response probability.
- **Customer Segmentation (Unsupervised – K-Means Clustering)**
Grouped retail customers into distinct clusters based on purchasing behavior for targeted marketing.
- **Movie Recommendation System (Unsupervised – K-Means)**
Built a recommendation engine using clustering & similarity measures.
- **Loan Default Prediction (Ensemble – Random Forest & Gradient Boosting)**
Boosted model accuracy by combining multiple classifiers to predict loan repayment defaults.
- **Sentiment Analysis Enhancement (Ensemble – Bagging & Boosting)**
Improved sentiment classification accuracy by combining weak learners into a strong predictive model.

Tech Stack: Python, Pandas, Scikit-learn, NumPy, Matplotlib

GitHub Repository: github.com/username/-----

Deep Learning Projects

Image Classification using CNN (MNIST Dataset)

Description: Built and trained a Convolutional Neural Network (CNN) model to classify handwritten digits (0–9) from the MNIST dataset. Achieved >98% accuracy through optimized architecture and hyperparameter tuning.

- Preprocessed dataset with normalization & reshaping for efficient training.
- Designed CNN with convolutional, pooling, dropout, and fully connected layers for feature extraction and classification.
- Applied ReLU activation for non-linearity and Softmax for multi-class output.
- Implemented Adam optimizer and categorical cross-entropy loss to improve convergence.
- Evaluated model using accuracy score, confusion matrix, and loss curves.
- Visualized predictions with Matplotlib to compare correctly vs. misclassified digits.

Tech Stack: Python, TensorFlow, Keras, NumPy, Matplotlib

GitHub Repository: github.com/username/-----

Real-time Emotion Detection (CNN + OpenCV)

Description: Developed a real-time facial emotion recognition system using Convolutional Neural Networks (CNNs) integrated with OpenCV for live video capture. The model successfully detects and classifies human emotions such as happy, sad, angry, surprised, and neutral from webcam input.

- Preprocessed facial images using grayscale conversion, resizing, and normalization for consistent model input.
- Designed and trained a CNN architecture with convolutional, pooling, dropout, and dense layers for robust emotion classification.
- Implemented Softmax activation to classify multiple emotion categories.
- Integrated OpenCV Haar Cascade & DNN face detectors to capture real-time faces from live webcam stream.
- Displayed emotion labels on video frames with bounding boxes for clear visualization.
- Achieved high accuracy on validation data and optimized inference speed for real-time deployment.

Tech Stack: Python, TensorFlow, Keras, OpenCV, NumPy, Matplotlib

GitHub Repository: github.com/username/-----

Text Sentiment Analysis using RNN/LSTM

Description: Built a Recurrent Neural Network (RNN) with LSTM layers to predict sentiment (positive/negative/neutral) from text data such as movie reviews and social media posts. The model effectively captured long-term dependencies in text for improved sentiment classification.

- Collected and preprocessed textual dataset with tokenization, stopword removal, padding, and embedding.
- Used Word Embeddings (Word2Vec/Glove/Embedding layer) to represent words in dense vector space.
- Designed an LSTM-based deep learning model capable of handling sequential dependencies in text.
- Implemented dropout layers to prevent overfitting and improve generalization.
- Trained the model with Adam optimizer & binary/multiclass cross-entropy loss.

Tech Stack: Python, TensorFlow, Keras, NLTK, NumPy, Matplotlib

GitHub Repository: github.com/username/-----

Generative AI Projects

FLAN-T5 Summarizer & Q&A Assistant

Description : Built a CPU-friendly CLI app that performs abstractive text summarization and context-aware Q&A using google/flan-t5-small—ideal for classroom demos and quick local workflows (no training required).

- Implemented two modes: Summarize text and Q&A from local notes (context.txt) with a strict "Not found" fallback if answers aren't in context.
- Used Hugging Face Transformers (Seq2Seq) with clean prompt engineering, safe generation (top-p, temperature), and length controls.
- Designed a simple, cross-platform CLI UX with clear instructions and modular code for students.
- Runs fully offline on CPU (Torch), no external APIs; fast first-token latency for short inputs.
- Demonstrates practical LLM skills: tokenization (SentencePiece), decoding, inference, and context grounding without fine-tuning.

Tech Stack: Python, Hugging Face Transformers, PyTorch, FLAN-T5, SentencePiece

GitHub Repository: github.com/username/-----

Achievements

- Completed Python – Machine Learning with Automations, Deep Learning & GenAI training with multiple projects in Marvellous Infosystems.
- Built strong GitHub portfolio with 20+ AI/ML projects.
- Conducted hands-on coding in real-time case studies and automation tasks.

Work history : (If applicable)

Experience : __ Years

Company name : _____

Domain Knowledge : _____

Responsibilities : _____

Profiles

- GitHub: github.com/username
- LinkedIn: linkedin.com/in/username

Personal Information:

- Date of Birth:
- Father's Name:
- Marital Status:
- Nationality:

The above mentioned information is authentic to the best of my knowledge.

Your Name